Fondements de la programmation

Informations générales

- 1h30 CM + 1h30 TD pendant 12 semaines
- Auteurs : Pierre Boudes, Flavien Breuvart, Paulin Jacobé de Naurois, Sergueï Lenglet
  `serguei.lenglet@lipn.univ-paris13.fr`

Évaluation

- Partiel (30 octobre)
- Examen (18 décembre)

Sources

- *Practical Foundations of Programming Languages*, Robert Harper
- *Types and Programming Languages*, Benjamin Pierce

Des comportements inattendus

- Programme
- Langage : pas ou mal défini
- Compilateur : optimisations
- Matériel : optimisations, concurrence

Des comportements inattendus

- Programme
  - spécifications
  - preuves de programme
- Langage : pas ou mal défini
  - outils : systèmes de types
  - langages sûrs par construction
- Compilateur : optimisations
  - CompCert
  - Verified LLVM
- Matériel : optimisations, concurrence
  - Modèle de mémoire faible
  - Certification

Des comportements inattendus

- Programme
  - spécifications
  - preuves de programme
- Langage : pas ou mal défini
  - outils : systèmes de types
  - langages sûrs par construction
- Compilateur : optimisations
  - CompCert
  - Verified LLVM
- Matériel : optimisations, concurrence
  - Modèle de mémoire faible
  - Certification

Grammaire

Langage de programmation

Langage naturel : infinité d'énoncés à partir d'ensembles finis de

- mots
- règles de grammaire

En programmation : syntaxe

- tokens : mots-clefs, identifiants, opérateurs, ...
- grammaire formelle

Exemple

Type			Expression		
$\tau ::=$	`Num`	(nombres)	$e ::=$	x	(variable)
	$\tau_1 \to \tau_2$	(fonctions)		n	(nombre littéral)
	$(\tau_1, \ldots, \tau_n)$	(produit)		$e_1 + e_2$	(ajouter)
	`Unit`	(produit vide)		$e_1 * e_2$	(multiplier)
				$(e_1, \ldots, e_k)$	(tuple)
				`let` x=e_1 `in` e_2	(définition)

Littéraux

Variables	$x = [a-z]^+$
Nombres	$n = [0-9]^+$

Terminaux, non-terminaux, règles

Type $\tau ::=$	`Num`	(nombres)
	$\tau_1 \to \tau_2$	(fonctions)
	$(\tau_1, \ldots, \tau_n)$	(produit)
	`Unit`	(produit vide)

- Un type peut être
  - `Num`
  - un type suivi de $\to$ et d'un type
  - (type, ..., type) (... pas formel mais toléré ici)
  - `Unit`
- Ou : si τ_1 et τ_2 sont des types, alors $\tau_1 \to \tau_2$ est un type

Terminaux, non-terminaux, règles

Type $\tau ::=$	`Num`	(nombres)
	$\tau_1 \to \tau_2$	(fonctions)
	$(\tau_1, \ldots, \tau_n)$	(produit)
	`Unit`	(produit vide)

- Un type peut être
  - `Num`
  - un type suivi de $\to$ et d'un type
  - (type, ..., type) (... pas formel mais toléré ici)
  - `Unit`
- Ou : si τ_1 et τ_2 sont des types, alors $\tau_1 \to \tau_2$ est un type
- Non-terminal : τ
- Terminaux : opérateurs `Num`, $\to$, *etc* mais aussi littéraux (variables x, nombres n, *etc*)
- Règle : d'un non-terminal vers terminaux/non-terminaux

Grammaire, définition formelle

Definition

Une grammaire est donnée par :

- un ensemble $\mathscr{T}$ de terminaux,
- un ensemble $\mathscr{N}$ de non-terminaux,
- un non-terminal principal $S \in \mathscr{T}$,
- une relation $\mapsto \subseteq \mathscr{N} \times (\mathscr{T} \cup \mathscr{N})^*$ décrivant les règles.

Grammaire, exemple

Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x=e_1 `in` e_2	(définition)

- Combien y-a-t-il de règles ?

Grammaire, exemple

Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x`=`e_1 `in` e_2	(définition)

- Combien y-a-t-il de règles ? 5
- Combien y-a-t-il de non-terminaux ?

Grammaire, exemple

Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x`=`e_1 `in` e_2	(définition)

- Combien y-a-t-il de règles ? 5
- Combien y-a-t-il de non-terminaux ? 1 : e
- Combien y-a-t-il de terminaux ?

Grammaire, exemple

Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x`=`e_1 `in` e_2	(définition)

- Combien y-a-t-il de règles ? 5
- Combien y-a-t-il de non-terminaux ? 1 : e
- Combien y-a-t-il de terminaux ? 7 : x, n, $+$, $*$, `let`, `=`, `in`
- Est-ce que `let` x`=1` `in` x`+2` est une expression valide ?

Grammaire, exemple

Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x`=`e_1 `in` e_2	(définition)

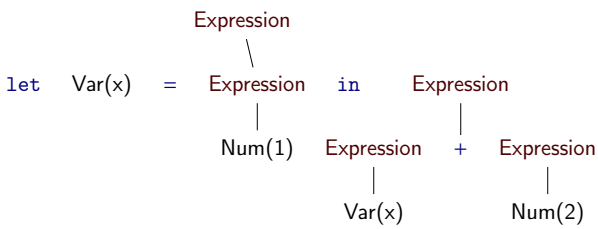
- Combien y-a-t-il de règles ? 5
- Combien y-a-t-il de non-terminaux ? 1 : e
- Combien y-a-t-il de terminaux ? 7 : x, n, $+$, $*$, `let`, `=`, `in`
- Est-ce que `let` x`=1` `in` x`+2` est une expression valide ? oui
- Est-ce que `let` 2`*`3`=`y `in` y est une expression valide ?

Grammaire, exemple

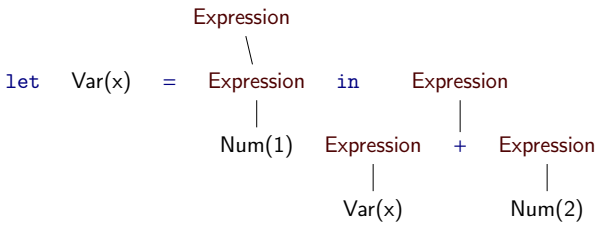
Expression $e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(ajouter)
	$e_1 * e_2$	(multiplier)
	`let` x`=`e_1 `in` e_2	(définition)

- Combien y-a-t-il de règles ? 5
- Combien y-a-t-il de non-terminaux ? 1 : e
- Combien y-a-t-il de terminaux ? 7 : x, n, $+$, $*$, `let`, `=`, `in`
- Est-ce que `let` x`=1` `in` x`+2` est une expression valide ? oui
- Est-ce que `let` 2`*`3`=`y `in` y est une expression valide ? non

Arbre syntaxique



Arbre syntaxique

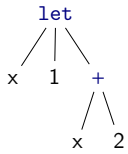


- Les nœuds internes sont des non-terminaux.
- Les feuilles sont des terminaux.
- Les fils d'un non-terminal N correspondent à une règle sur N.

Arbre syntaxique abstrait (*Abstract Syntax Tree*, AST)

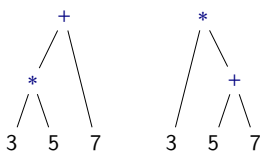
Simplification arbitraire de l'arbre syntaxique :

- suppression des parenthèses
- les nœuds internes sont étiquetés avec le nom de la règle utilisée (ou directement l'opérateur si pas d'ambiguïté)
- cf cours de compilation

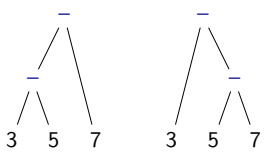


Problèmes d'ambguïté

Priorité : 3 * 5 + 7



Associativité : 3 - 5 - 7



Problèmes d'ambguïté

Priorité : 3 * 5 + 7



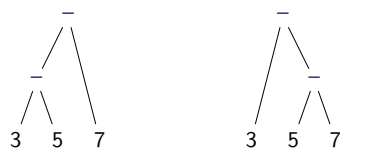
* prioritaire sur + + prioritaire sur *

Solutions :

- Changer la grammaire
- Indiquer les priorités (dans ce cours)

Problèmes d'ambguïté

Associativité : 3 - 5 - 7



- associatif à gauche - associatif à droite

Solutions :

- Changer la grammaire
- Indiquer les associativités (dans ce cours)

Problèmes d'ambguïté

Pas seulement pour les operateurs binaires :

- `!b==c` si `!` prioritaire sur `==`

Problèmes d'ambguïté

Pas seulement pour les operateurs binaires :

- `!b==c` si `!` prioritaire sur `==` `(!b)==c`
- `if b then if c then ; else ;` si `if` ternaire prioritaire sur `if` binaire

Problèmes d'ambguïté

Pas seulement pour les operateurs binaires :

- `!b==c` si `!` prioritaire sur `==` `(!b)==c`
- `if b then if c then ; else ;` si `if` ternaire prioritaire sur `if` binaire if b then (if c then ; else ;)
- `true?false:true?true:true` si `_?_:_` associatif à droite

Problèmes d'ambguïté

Pas seulement pour les operateurs binaires :

- `!b==c` si `!` prioritaire sur `==` `(!b)==c`
- `if b then if c then ; else ;` si `if` ternaire prioritaire sur `if` binaire if b then (if c then ; else ;)
- `true?false:true?true:true` si `_?_:_` associatif à droite true ?false :(true ?true :true)

Portée d'une variable, lieur

Quel est le résultat de `let` x=1 `in let` x=2 `in` x+x

Portée d'une variable, lieur

Quel est le résultat de `let` x=1 `in let` x=2 `in` x+x 4

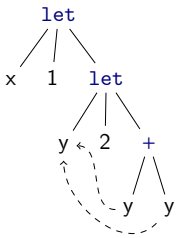
Portée d'une variable, lieur

Quel est le résultat de `let` x=1 `in let` x=2 `in` x+x 4

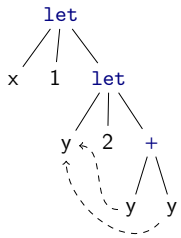
Peut s'écrire aussi `let` x=1 `in let` y=2 `in` y+y

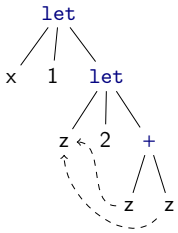
x et y sont des variables liées

Arbre lieur abstrait (*Abstract Binding Tree* ABT)



Arbre lieur abstrait (*Abstract Binding Tree* ABT)





Pas autant utilisés que les AST

17 / 17